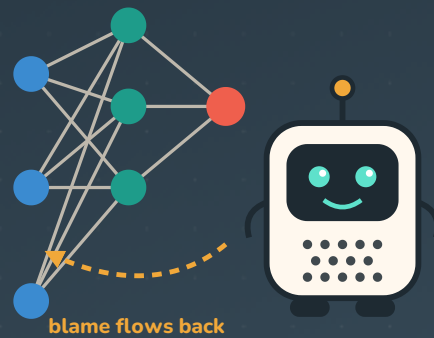


A Real Brain

Neurons in layers. Run them forwards with matrices;
send the blame backwards through the same wires.
The puzzle from Book 2 falls, the walls learn to bend,
and Nova finally hears its name.

Inside: XOR solved by a hidden layer · why a network must bend (and the three-line proof that stacked straight lines stay straight) · sigmoid, tanh, ReLU · forward propagation as matrix multiplication, shapes and all · backpropagation derived link by link, then vectorised, then worked by hand · the update, random starts, and XOR learned from scratch · depth versus width on two moons · a network in forty lines · Nova, at last



$$\mathbf{D}^1 = (\mathbf{D}^2 \mathbf{W}^2 \mathbf{r}) \odot \mathbf{g}'(\mathbf{Z}^1)$$

Where we are in the story

THE FOUR BOOKS SO FAR

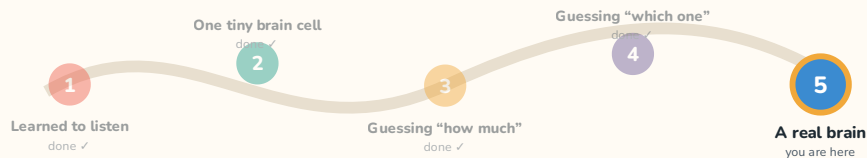
Book 1: show, don't tell — four ingredients, one loop. **Book 2:** one neuron is a straight wall; it learns anything one wall can cut, and XOR is not one of those things. **Book 3:** take off the cliff, score with squared error, feel the slope ($2e \cdot x$), walk downhill. **Book 4:** squash with a sigmoid, score with surprise, and the slope at the output is simply $p - y$; step one example at a time or in mini-batches; softmax for many classes.

Every one of those neurons was a single layer. Every wall was straight. Every gradient was $\text{error} \times \text{input}$, found by multiplying the links of a chain.

THIS TIME

We put a layer of neurons *between* the inputs and the output. The chain gets longer, and the multiply-the-links rule does the rest: the output's blame $p - y$ is passed backwards through the same wires, each hidden neuron receives its share, and every knob in the network gets its own gradient. That is **backpropagation** — the thing Rosenblatt's rule could not do.

With it, XOR is learned from random knobs in a few hundred steps; a network of bent walls separates two crescent moons; and the whole story closes where it began, with Nova listening for its name.



What you need

Books 3 and 4: the chain rule as "multiply the links", and $p - y$ at the output. One new tool: writing many neurons at once as a matrix multiplication. If you have met matrices, good; if not, Chapter 3 shows exactly what each entry means, with the numbers. The companion notebook ([Book5_MLP_From_Scratch.ipynb](#)) contains a general network in forty lines, plus a *gradient check* that verifies backpropagation against nudging every knob by hand.

The boxes you will meet

Story · Big idea · Think of it like... · Stop and think · Our answer · Try it yourself · Careful · Spark · Words to know · numbered derivation boxes · and, new, matrix boxes showing every number in a forward or backward pass.

The wall, revisited

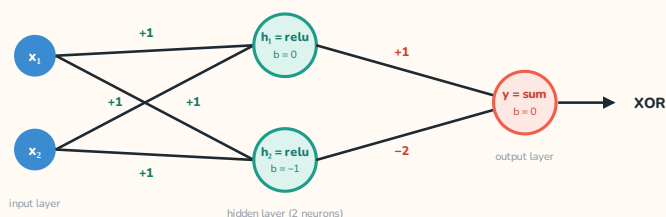
XOR beat one neuron in Book 2. Here it falls to three — and the way it falls is the whole idea of a hidden layer.

STORY

Asha opens the notebook from Book 2. The last page still says: *XOR — no single line works — the rule cannot train a hidden layer*. Nova has since learned to say numbers, probabilities and votes. But every wall it draws is still straight, and the wake word — the reason all this began — is not a straight-wall problem. “Time,” says Asha, “to finish what we started.”

Three neurons, two layers, and XOR is solved

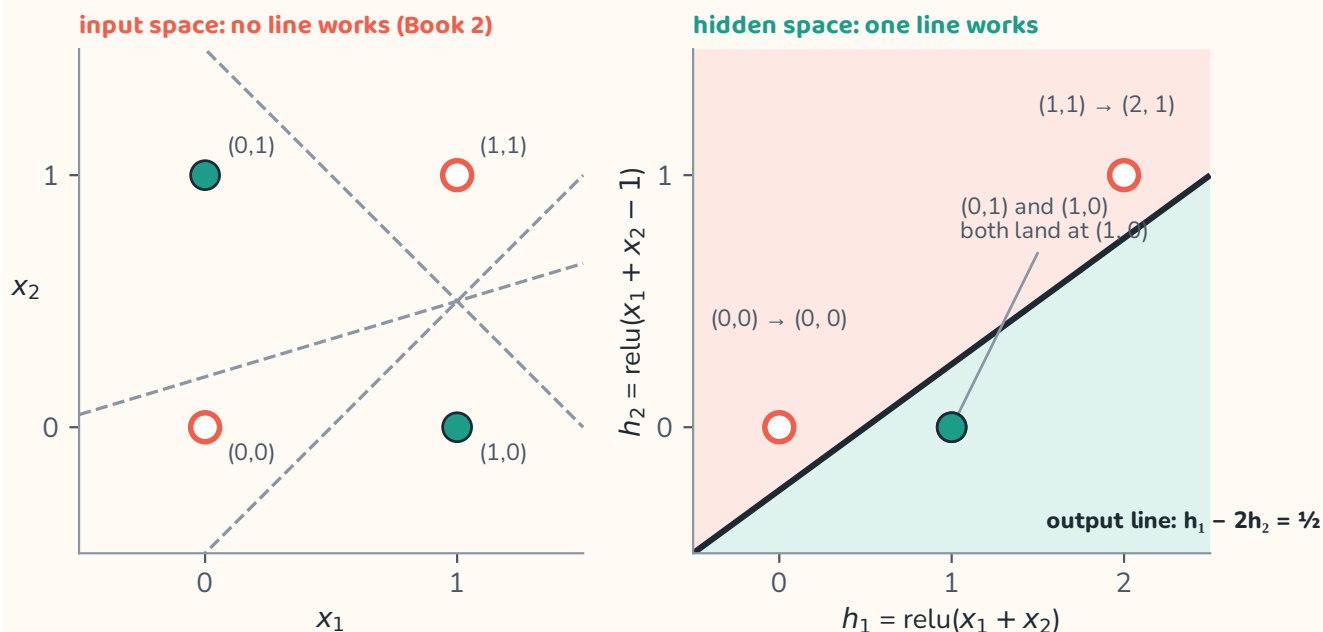
In Book 2 we built XOR from an OR, a NAND and an AND. Here is an even simpler version with the modern activation function **ReLU** (“rectified linear unit”), which just means: *keep the number if it is positive, otherwise output 0*. Two hidden neurons, one output neuron, and every knob is a small whole number:



A 2-2-1 network. $h_1 = \text{relu}(x_1 + x_2)$, $h_2 = \text{relu}(x_1 + x_2 - 1)$, $y = h_1 - 2h_2$. Every knob is +1, -1, -2 or 0.

x_1, x_2	$x_1 + x_2$	$h_1 = \text{relu}(x_1 + x_2)$	$x_1 + x_2 - 1$	$h_2 = \text{relu}(x_1 + x_2 - 1)$	$y = h_1 - 2h_2$	wanted
0, 0	0	0	-1	0	0	0 ✓
0, 1	1	1	0	0	1	1 ✓
1, 0	1	1	0	0	1	1 ✓
1, 1	2	2	1	1	0	0 ✓

Read the trick. h_1 counts how many inputs are on. h_2 counts how many *beyond the first* are on — it only wakes up when both are. The output says “ h_1 , minus a double penalty if h_2 woke up”. One input on: $1 - 0 = 1$. Both on: $2 - 2 = 0$. The second hidden neuron is a “both?” detector, and that is exactly the feature XOR needed — the $x_1 x_2$ that we had to invent by hand in Book 2, now built by a neuron.



What the hidden layer did. Left: the four XOR points where no straight line works. Right: the same four points, plotted by what the two hidden neurons output. (0,1) and (1,0) land on the same spot, and the two 0-cases are pulled apart. Now one line — the output neuron — separates them. **A hidden layer re-draws the map until a straight wall is enough.**

💡 BIG IDEA — THE ANATOMY OF A NETWORK

A **multi-layer perceptron** (MLP), or **deep feedforward network**, is neurons arranged in layers: an **input layer** (just the numbers), one or more **hidden layers**, and an **output layer**. Every neuron in a layer takes *all* the outputs of the layer before, computes a weighted sum plus bias, and applies an activation. Nothing loops back; the numbers only flow forward — hence *feedforward*. The number of hidden layers is the network's **depth**; the number of neurons in a layer is its **width**. Book 1 called the layers a stack. This is the stack.

The names we will use

Symbol	Meaning	In the XOR network above
$\mathbf{X}$	the inputs, one example per row	4 rows (the four cases) $\times$ 2 columns
$\mathbf{W}^1, \mathbf{b}^1$	knobs of the hidden layer: $\mathbf{W}^1$ has one column per hidden neuron	$\mathbf{W}^1$ is 2×2 , all $+1$; $\mathbf{b}^1 = (0, -1)$
$\mathbf{Z}^1 = \mathbf{X}\mathbf{W}^1 + \mathbf{b}^1$	the hidden layer's weighted sums ("pre-activations")	4×2
$\mathbf{A}^1 = g(\mathbf{Z}^1)$	the hidden layer's outputs, after the activation g	$4 \times 2, g = \text{relu}$
$\mathbf{W}^2, \mathbf{b}^2$	knobs of the output layer	$\mathbf{W}^2 = (1, -2)^T, b^2 = 0$
$\mathbf{Z}^2 = \mathbf{A}^1\mathbf{W}^2 + \mathbf{b}^2$	the output's weighted sum	4×1
$\mathbf{P} = \sigma(\mathbf{Z}^2)$	the output, squashed (when we want a probability)	4×1

Superscripts on $\mathbf{W}, \mathbf{b}, \mathbf{Z}, \mathbf{A}$ count layers (they are not powers). We will use a sigmoid output and the surprise loss from Book 4 whenever the network is trained — the hand-built network above skips the squash because its output is already 0 or 1.

⚠️ CAREFUL — WE SET THESE KNOBS BY HAND

Nobody learned $+1, +1, -1, -2$. We chose them, as we chose OR and NAND in Book 2. The question of this book is not *whether* a hidden layer can solve XOR — it can — but how a network can *find* such knobs from the four examples alone. Chapters 5 and 6 answer that. First we need to understand why the hidden layer must bend.

Why a network must bend

Stack ten straight lines and you still have a straight line. The activation function is what lets a network draw a curve — and the proof takes three lines.

STOP AND THINK

Suppose the hidden layer had *no* activation — each hidden neuron just passed on its weighted sum z , like the regression neuron of Book 3. Then the output takes a weighted sum of weighted sums. Is that more powerful than a single neuron?

OUR ANSWER — NO, AND HERE IS THE PROOF

1 $\mathbf{h} = \mathbf{x}\mathbf{W}^1 + \mathbf{b}^1$

2 $y = \mathbf{h}\mathbf{W}^2 + b^2 = (\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1)\mathbf{W}^2 + b^2$

3 $y = \mathbf{x}\underbrace{\mathbf{W}^1\mathbf{W}^2}_{\tilde{\mathbf{W}}} + \underbrace{\mathbf{b}^1\mathbf{W}^2 + b^2}_{\tilde{b}} = \mathbf{x}\tilde{\mathbf{W}} + \tilde{b}$

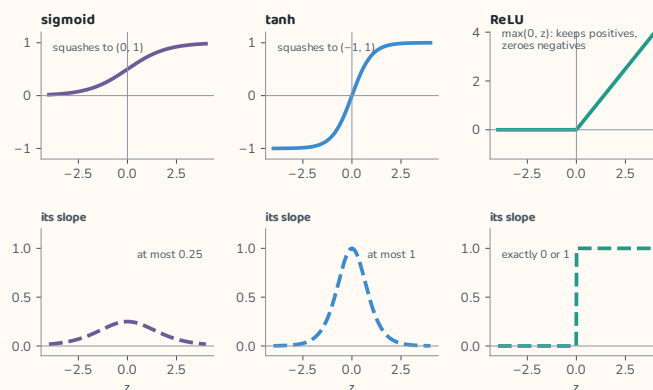
hidden layer *without* activation: a straight-line function of $\mathbf{x}$ (for one neuron, $h = w_1x_1 + w_2x_2 + b$)

output layer: a straight-line function of $\mathbf{h}$; substitute $\mathbf{h}$

multiply out and group: **one** straight-line function of $\mathbf{x}$, with knobs $\tilde{\mathbf{W}}$ and $\tilde{b}$. A single neuron could have had those knobs directly. Two layers of straight lines collapse into one — and so do ten.

So a hidden layer is only useful if something **non-linear** happens inside it — something that is *not* a weighted sum. That something is the **activation function**, and its job is to bend.

Three ways to bend

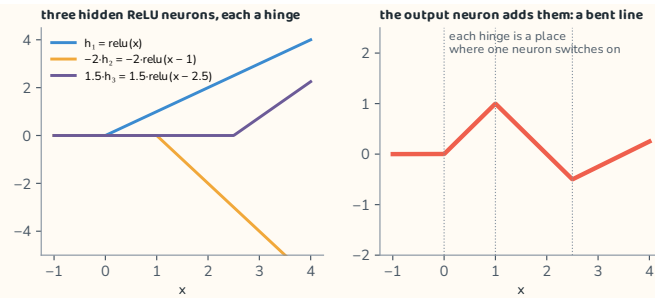


The three activations you will meet, with their slopes. The slope matters for backpropagation (Chapter 5): it is one of the links in the chain. Sigmoid's slope is at most 0.25 and tanh's at most 1 — both fade to zero for large inputs. ReLU's slope is exactly 1 where the neuron is on and 0 where it is off.

Name	Formula	Output range	Slope $g'(z)$	Notes
sigmoid	$\frac{1}{1 + e^{-z}}$	$(0, 1)$	$g(z)(1 - g(z))$	Book 4's squash; still the standard <i>output</i> for yes/no. Slope small everywhere — slow to train in deep stacks.
tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	$1 - g(z)^2$	A sigmoid stretched and centred at 0. Good for small networks; we use it to learn XOR.
ReLU	$\max(0, z)$	$[0, \infty)$	1 if $z > 0$, else 0	The modern default for hidden layers: cheap, slope never fades when on. A neuron that is off passes nothing forward and no blame back.

How bends become curves

Look at one ReLU neuron with one input: $\text{relu}(x - c)$ is flat until x reaches c , then rises like a ramp. A *hinge*. Now give the output neuron three such hinges to add up, each with its own weight:



Hinges add up to a bent line. Each hidden ReLU neuron contributes one kink, at the place where it switches on. The output neuron's weights set how sharp each kink is and which way it goes. With enough hinges you can trace any curve as closely as you like. In two dimensions each hidden neuron is a fold in the sheet; the output cuts the folded sheet with one straight line.

∞ THINK OF IT LIKE... FOLDING PAPER BEFORE CUTTING

You cannot cut a star out of paper with one straight snip of the scissors. But fold the paper a few times first, make one straight cut, unfold — and out comes a star. The hidden layer *folds* the input space (each ReLU neuron is one fold, along the line where its sum is zero). The output neuron makes *one straight cut* through the folded sheet. Unfold, and the cut becomes a bent boundary in the original space. That is why a network of straight-wall neurons can draw curves: it cuts straight in a folded world.

✧ SPARK — ANY SHAPE AT ALL

In 1989 it was proved that a network with just *one* hidden layer, given enough neurons, can match any reasonable function as closely as you wish — the **universal approximation theorem**. Enough hinges trace any curve; enough folds cut any shape. The catch is in “enough”: one wide layer may need an absurd number of neurons where a few stacked layers manage with a handful, because a deeper network can *reuse* its folds. That is the case for depth, and Chapter 7 shows it with pictures.

🧪 TRY IT YOURSELF

On paper, with one input x : design two ReLU neurons and an output weight for each so that the output is 0 for $x < 1$, rises to 1 at $x = 2$, and stays at 1 afterwards. (Hint: a ramp that starts at 1, minus a ramp that starts at 2.) Check your answer upside down:

$$y = \text{relu}(x) - \text{relu}(x - 1) + \text{relu}(x - 2)$$

📖 WORDS TO KNOW

Non-linearity — any operation that is not a weighted sum; what makes hidden layers worth having. **Activation function g** — the per-neuron bend: sigmoid, tanh, ReLU. **ReLU** — $\max(0, z)$. **Pre-activation z** — the weighted sum before g . **Universal approximation** — enough hidden neurons can fit any curve.

3

Forward propagation: plain maths, then matrices

First one neuron at a time, with nothing but weighted sums. Then every neuron in a layer, for every example in the batch, in one multiplication — which is how a network is actually run. Every number is shown.

PLAIN MATHS — one example, one neuron at a time

Take one example $\mathbf{x} = (x_1, x_2)$. Hidden neuron j has its own two weights w_{1j}, w_{2j} and bias b_j (the first index says which input, the second which neuron). It does what every neuron in this series has done: a weighted sum, then a bend. The output neuron does the same to the hidden outputs, then squashes.

$$1 \quad z_1 = w_{11}x_1 + w_{21}x_2 + b_1 \quad z_2 = w_{12}x_1 + w_{22}x_2 + b_2$$

Hidden totals. Two neurons, two weighted sums of the same inputs, each with its own knobs.

$$2 \quad a_1 = g(z_1) \quad a_2 = g(z_2)$$

Bend. g is ReLU, tanh or sigmoid (Chapter 2).

$$3 \quad z^{(2)} = v_1 a_1 + v_2 a_2 + c$$

Output total. The output neuron's weights v_1, v_2 and bias c act on the hidden *outputs*, not on $\mathbf{x}$.

$$4 \quad p = \sigma(z^{(2)})$$

Squash. A probability, as in Book 4. (The hand-built XOR network skips this because its total is already 0 or 1.)

The XOR network of Chapter 1 on the input (1, 1): all four hidden weights are 1, $b_1 = 0$, $b_2 = -1$, $v = (1, -2)$, $c = 0$:

$$z_1 = 1 \cdot 1 + 1 \cdot 1 + 0 = 2, \quad z_2 = 1 \cdot 1 + 1 \cdot 1 - 1 = 1, \quad a_1 = \text{relu}(2) = 2, \quad a_2 = \text{relu}(1) = 1, \quad z^{(2)} = 1 \cdot 2 + (-2) \cdot 1 + 0 = 0 \quad \checkmark$$

↓ now write the **two hidden neurons as one matrix**, then stack the **four examples as rows**

MATRIX FORM — every neuron, every example, one product per layer

Put neuron 1's weights in the first column of a matrix $\mathbf{W}^1$ and neuron 2's in the second. Then step 1 for *both* neurons is one row-times-matrix product. Stack the examples as the rows of $\mathbf{X}$, and the same product does every example too. Superscripts count layers; they are not powers.

$$1 \quad \mathbf{Z}^1 = \mathbf{X}\mathbf{W}^1 + \mathbf{b}^1 \quad (n \times h)$$

entry (i, j) = row i of $\mathbf{X}$ dotted with column j of $\mathbf{W}^1$, plus b_j : example i 's total at hidden neuron j . The bias row is added to every row ("broadcasting").

$$2 \quad \mathbf{A}^1 = g(\mathbf{Z}^1) \quad (n \times h)$$

the bend, applied to every entry

$$3 \quad \mathbf{Z}^2 = \mathbf{A}^1\mathbf{W}^2 + \mathbf{b}^2 \quad (n \times k)$$

the output layer is the same product with $\mathbf{A}^1$ as its input; $\mathbf{W}^2$ holds v as a column

$$4 \quad \mathbf{P} = \sigma(\mathbf{Z}^2) \quad (n \times k)$$

one probability per example (or a softmax row for many classes)

Four examples, two layers, three matrix products, no loops. A batch of ten thousand images through a network of ten layers is the same picture with bigger rectangles.

The XOR network, every number

The hand-built network of Chapter 1, run on all four XOR inputs at once. Follow one entry: row 4 of $\mathbf{X}$ is (1, 1); column 2 of $\mathbf{W}^1$ is (1, 1); their dot product is 2, plus $b_2 = -1$, gives $Z_{4,2}^1 = 1$ — the same $z_2 = 1$ as in the plain rung above.

$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}_{4 \times 2} \quad \mathbf{W}^1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \mathbf{b}^1 = [0 \quad -1] \quad \mathbf{W}^2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \quad \mathbf{b}^2 = [0]$$

$$\mathbf{Z}^1 = \mathbf{X}\mathbf{W}^1 + \mathbf{b}^1 = \begin{bmatrix} 0+0+0 & 0+0-1 \\ 0+1+0 & 0+1-1 \\ 1+0+0 & 1+0-1 \\ 1+1+0 & 1+1-1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} \quad \mathbf{A}^1 = \text{relu}(\mathbf{Z}^1) = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

$$\mathbf{Z}^2 = \mathbf{A}^1\mathbf{W}^2 + \mathbf{b}^2 = \begin{bmatrix} 0 \cdot 1 + 0 \cdot (-2) \\ 1 \cdot 1 + 0 \cdot (-2) \\ 1 \cdot 1 + 0 \cdot (-2) \\ 2 \cdot 1 + 1 \cdot (-2) \end{bmatrix} + 0 = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad \leftarrow \text{XOR, all four at once}$$

rows are examples, columns are neurons — read every matrix in this book that way

The shapes, in general

Let n be the batch size, d the number of inputs, h the number of hidden neurons, and k the number of outputs ($k = 1$ for yes/no, $k = 3$ for three flowers). The rule for every layer is the same: *rows are examples, columns are neurons*.

Object	Shape	Read it as
$\mathbf{X} = \mathbf{A}^0$	$n \times d$	n examples, d inputs each
$\mathbf{W}^1, \mathbf{b}^1$	$d \times h, \quad 1 \times h$	each column: one hidden neuron's d weights; one bias per neuron
$\mathbf{Z}^1 = \mathbf{A}^0 \mathbf{W}^1 + \mathbf{b}^1, \quad \mathbf{A}^1 = g(\mathbf{Z}^1)$	$n \times h$	every example's total at every hidden neuron; then bent
$\mathbf{W}^2, \mathbf{b}^2$	$h \times k, \quad 1 \times k$	each column: one output neuron's h weights
$\mathbf{Z}^2 = \mathbf{A}^1 \mathbf{W}^2 + \mathbf{b}^2, \quad \mathbf{P} = \sigma(\mathbf{Z}^2)$	$n \times k$	every example's output score; then squashed to a probability

Sanity check for any matrix product: the inner numbers must match ($n \times d$ times $d \times h$) and the result takes the outer ones ($n \times h$). If the shapes do not line up, the network is wired wrong. Half of all bugs in practice are shape bugs.

💡 THE FORWARD ALGORITHM, FOR ANY DEPTH

Input	$\mathbf{A}^0 = \mathbf{X}$ (the batch). Knobs $\mathbf{W}^1 \dots \mathbf{W}^L, \mathbf{b}^1 \dots \mathbf{b}^L$.
For each layer $l = 1 \dots L$	$\mathbf{Z}^l = \mathbf{A}^{l-1} \mathbf{W}^l + \mathbf{b}^l$, then $\mathbf{A}^l = g(\mathbf{Z}^l)$ for hidden layers, or $\mathbf{A}^L = \sigma(\mathbf{Z}^L)$ for the output.
Keep	every $\mathbf{Z}^l$ and $\mathbf{A}^l$ — the backward pass will need them.
Output	$\mathbf{P} = \mathbf{A}^L$: one probability per example (or one row of softmax shares, for many classes).

🔄 THINK OF IT LIKE... AN ASSEMBLY LINE, REVISITED

Book 1 called a deep network an assembly line. Now you can see the line exactly: each station (layer) receives a tray of parts (a matrix, one row per product), does one identical operation on every row (multiply by $\mathbf{W}$, add $\mathbf{b}$, bend), and passes the tray on. The stations do not need to know what the products are. And because every row gets the same treatment, a machine that multiplies matrices can run the whole tray at once — which is precisely what a GPU is.

🔧 TRY IT YOURSELF

A network takes 30 inputs (the tumour measurements), has a hidden layer of 8 neurons and one sigmoid output, and is fed a mini-batch of 16 patients. Write the shapes of $\mathbf{X}, \mathbf{W}^1, \mathbf{b}^1, \mathbf{Z}^1, \mathbf{A}^1, \mathbf{W}^2, \mathbf{b}^2, \mathbf{Z}^2, \mathbf{P}$. How many knobs does the network have in total? Check upside down:

knobs = $240 + 8 + 8 + 1 = 257$
 $\mathbf{X} \text{ } 16 \times 30, \mathbf{W}^1 \text{ } 30 \times 8, \mathbf{b}^1 \text{ } 1 \times 8, \mathbf{Z}^1 \text{ and } \mathbf{A}^1 \text{ } 16 \times 8, \mathbf{W}^2 \text{ } 8 \times 1, \mathbf{b}^2 \text{ } 1 \times 1, \mathbf{Z}^2 \text{ and } \mathbf{P} \text{ } 16 \times 1$

⚠️ CAREFUL — WHICH WAY ROUND?

We write $\mathbf{XW}$ with examples as rows and neuron-weights as columns, because that is how NumPy and most code do it. Some textbooks write $\mathbf{Wx}$ with the roles transposed. Both are the same arithmetic; only the picture is rotated. When reading a formula, ask “which index is the example, which is the neuron?” and the rest follows.

4

The loss, and the error at the output

Nothing new here — that is the point. The output layer is a Book 3 or Book 4 neuron, so its loss and its blame are ones you already know.

A network can end in any of the three output neurons we have built, and each brings its own loss. In every case, the quantity we need for training is the **blame at the output's weighted sum** — how the loss changes if z^L nudges — and in every case it is a difference between the guess and the truth:

Task	Output layer	Loss (per example)	Blame at the output, $\partial L / \partial z^L$	From
how much?	$\hat{y} = z^L$ (no squash)	$(\hat{y} - y)^2$	$2(\hat{y} - y)$ — or $\hat{y} - y$ with $\frac{1}{2}(\hat{y} - y)^2$	Book 3
yes / no?	$p = \sigma(z^L)$	$-[y \log p + (1 - y) \log(1 - p)]$	$p - y$	Book 4
which of K ?	$\mathbf{p} = \text{softmax}(\mathbf{z}^L)$	$-\log p_{\text{true}}$	$\mathbf{p} - \mathbf{y}$ (one-hot $\mathbf{y}$)	Book 4

$$\mathbf{D}^L = \frac{1}{n} (\mathbf{P} - \mathbf{Y}) \quad (n \times k)$$

for a batch, stack the blames one row per example — the same shape as $\mathbf{Z}^L$ — and divide by n , because the loss is an average. We use the yes/no case throughout.

The hand-built XOR network, scored honestly

Put a sigmoid on the output of Chapter 1's network and score it with surprise. Its scores $\mathbf{Z}^2$ were (0, 1, 1, 0), so:

$$\mathbf{Z}^2 = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad \mathbf{P} = \sigma(\mathbf{Z}^2) = \begin{bmatrix} 0.500 \\ 0.731 \\ 0.731 \\ 0.500 \end{bmatrix} \quad \mathbf{Y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

$$\text{surprise per example: } -\log(1 - 0.5) = 0.693, \quad -\log 0.731 = 0.313, \quad 0.313, \quad 0.693 \quad \Rightarrow \quad L = \frac{0.693 + 0.313 + 0.313 + 0.693}{4} = 0.503$$

$$\mathbf{P} - \mathbf{Y} = \begin{bmatrix} +0.500 \\ -0.269 \\ -0.269 \\ +0.500 \end{bmatrix} \quad \leftarrow \text{the blame, one number per example}$$

Every *decision* is right ($p \geq 0.5$ exactly when XOR = 1), yet the loss is 0.503 — the network is only mildly sure. The blame says what to do: push the output score down for the 0-cases (positive blame $\rightarrow$ decrease z^2) and up for the 1-cases. Training would scale $\mathbf{W}^2$ and b^2 up until the sigmoid is confident. Decisions can be perfect while the loss still has work to do; this is why we train on the loss, not on accuracy.

💡 BIG IDEA — THE OUTPUT'S BLAME IS THE SEED

$\mathbf{D}^L = (\mathbf{P} - \mathbf{Y})/n$ is one number per example, sitting at the output's weighted sum. It is the *only* place where the labels enter. Everything the hidden layers will learn comes from this seed being passed backwards, wire by wire, and multiplied by the slopes it meets on the way. The next chapter does the passing.

💡 STOP AND THINK

In the table above, why is the blame the *same shape* — guess minus truth — for three different losses and three different output neurons?

✓ OUR ANSWER

Because each loss was chosen to *match* its output neuron: squared error with the ramp, surprise with the sigmoid, surprise with the softmax. In each pairing the slope of the loss and the slope of the squash cancel (Book 4, Chapter 3). Mismatch them — squared error through a sigmoid, say — and the blame picks up an extra factor $p(1 - p)$ that fades when the neuron is confidently wrong. The pairings are not tradition; they are engineering.

📖 WORDS TO KNOW

Output blame $\mathbf{D}^L$ — $\partial L / \partial \mathbf{Z}^L$, guess minus truth (over n); the seed of backpropagation. Older books call it the output *delta*, δ^L . **Matched pair** — an output activation and a loss whose slopes cancel: ramp + squared error, sigmoid + surprise, softmax + cross-entropy.

Backpropagation: the blame flows back

Rosenblatt could not train a hidden layer because nobody could tell a hidden neuron how wrong it was. The chain rule can. It is the same multiply-the-links rule as Books 3 and 4 — the chain is just longer.

STORY

“The output neuron knows its error,” says Asha. “It is $p - y$. But look at hidden neuron two. It never sees the label. Did it help or hurt? By how much?” She stares at the wires. “Wait. The output neuron *listened* to it — through a wire with a weight. If I know how wrong the output was, and how hard it was listening, I know how much of the blame to pass back.” This is the question that stalled the field for seventeen years, and the answer that fits on one line.

One knob, deep inside

Take one example, a network with two hidden neurons and one sigmoid output, and pick a knob in the *hidden* layer: w_{ij} , the wire from input i to hidden neuron j . How does the loss change if we nudge it? Follow the path from the knob to the loss, and write down the slope of every link. Each link is something you already know. The output neuron’s weights are v_1, v_2 and its bias is c , as in Chapter 3.

PLAIN MATHS — one example, one hidden knob, four links

- 1 $w_{ij} \rightarrow z_j = \sum_i w_{ij}x_i + b_j \rightarrow a_j = g(z_j) \rightarrow z^{(2)} = \sum_j v_j a_j + c \rightarrow l$ **The path** from the knob to the loss has five steps.
- 2 $\frac{\partial L}{\partial z^{(2)}} = p - y$ **Link 4** — Book 4: the matched pair (sigmoid + surprise) gives guess minus truth.
- 3 $\frac{\partial z^{(2)}}{\partial a_j} = v_j$ **Link 3** — the slope of a weighted sum with respect to an *input* is the weight on that input: the wire.
- 4 $\frac{\partial a_j}{\partial z_j} = g'(z_j)$ **Link 2** — the activation’s slope (Chapter 2): 1 or 0 for ReLU, $1 - a_j^2$ for tanh.
- 5 $\frac{\partial z_j}{\partial w_{ij}} = x_i$ **Link 1** — Book 3: the slope of a weighted sum with respect to a *weight* is the input on that weight.
- 6 $\frac{\partial L}{\partial w_{ij}} = \underbrace{(p - y)}_{\text{link 4}} \cdot \underbrace{v_j}_{\text{link 3}} \cdot \underbrace{g'(z_j)}_{\text{link 2}} \cdot \underbrace{x_i}_{\text{link 1}}$ **Multiply the links.**
- 7 $\delta_j = (p - y) v_j g'(z_j) \implies \frac{\partial L}{\partial w_{ij}} = \delta_j x_i, \quad \frac{\partial L}{\partial b_j} = \delta_j$ **Name the middle.** Everything except the input is the **blame** δ_j of hidden neuron j . Then it is **blame $\times$ input** — the third time in three books, now for a neuron that never sees the label.

Many wires out. If the output layer has k neurons (three flowers), hidden neuron j feeds all of them, and its blame is the *sum* of what arrives on each wire, then bent by its slope:

$$\delta_j = \left(\delta_1^{(2)} v_{j1} + \delta_2^{(2)} v_{j2} + \cdots + \delta_k^{(2)} v_{jk} \right) g'(z_j) \quad \text{with} \quad \delta_m^{(2)} = p_m - y_m$$

Look at the bracket. It is row j of the output weight matrix $\mathbf{W}^2$ dotted with the vector of output blames — which is entry j of the row vector $\delta^{(2)} \mathbf{W}^{2T}$. Forwards, the signal went *through* $\mathbf{W}^2$; backwards, the blame comes *through* $\mathbf{W}^2$ *transposed*. The same wires carry both.

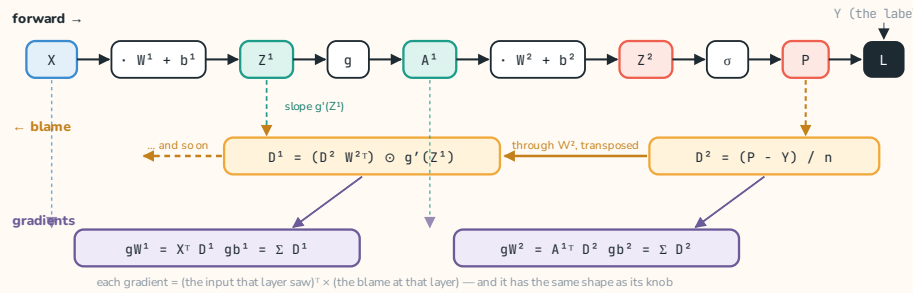
↓ the same, for **every neuron and every example** — rows are examples, columns are neurons

MATRIX FORM — the four lines of backpropagation

Stack the examples as rows, as in Chapter 3. $\mathbf{D}^2$ and $\mathbf{D}^1$ hold every example’s blame at every neuron of that layer; $\odot$ means multiply entry by entry.

- 1 $\mathbf{D}^2 = \frac{1}{n} (\mathbf{P} - \mathbf{Y}) \quad (n \times k)$ **the seed** — the output’s blame (Chapter 4), divided by n because the loss is an average
- 2 $\nabla_{\mathbf{W}^2} L = \mathbf{A}^{1T} \mathbf{D}^2 \quad (h \times k) \quad \nabla_{\mathbf{b}^2} L = \mathbf{1}^T \mathbf{D}^2$ **blame $\times$ input, added over the batch:** entry (j, m) of $\mathbf{A}^{1T} \mathbf{D}^2$ is $\sum_i a_{ij} \delta_{im}^{(2)}$; the bias gradient is the column sums
- 3 $\mathbf{D}^1 = (\mathbf{D}^2 \mathbf{W}^{2T}) \odot g'(\mathbf{Z}^1) \quad (n \times h)$ **pass the blame back** through the wires (transposed), then bend it by each hidden neuron’s slope — step 7 and the “many wires” sum, for everyone at once
- 4 $\nabla_{\mathbf{W}^1} L = \mathbf{X}^T \mathbf{D}^1 \quad (d \times h) \quad \nabla_{\mathbf{b}^1} L = \mathbf{1}^T \mathbf{D}^1$ **blame $\times$ input again** — Book 4’s $\mathbf{X}^T (\mathbf{P} - \mathbf{Y})$ with the hidden blame in place of $\mathbf{P} - \mathbf{Y}$

Check the shapes: $(n \times k)(k \times h) = n \times h$, the shape of $\mathbf{Z}^1$, so step 3 lines up; $(d \times n)(n \times h) = d \times h$, the shape of $\mathbf{W}^1$, so every gradient has the same shape as its knob. It must — we are going to subtract one from the other.



Three lanes. Forward: the batch flows left to right and every Z^l and A^l is kept. Blame: the seed $(P - Y)/n$ flows right to left, passing through each weight matrix transposed and being bent by each activation's slope. Gradients: at every layer, the input it saw (transposed) times the blame that reached it.

💡 BIG IDEA — BLAME IS PASSED BACK ALONG THE WIRE, THEN BENT BY THE SLOPE

A hidden neuron's blame has three factors: **the blame of the neuron it feeds** ($p - y$, which we called $\delta^{(2)}$ in Book 4), **the weight of the wire between them** (v_j : how hard the output was listening), and **the slope of its own activation** ($g'(z_j)$: how much its output would have moved). Once a neuron knows its blame, the gradient of each of its knobs is blame × input, exactly as before. **Backpropagation is nothing more than computing every neuron's blame, from the output backwards, one layer at a time.** The labels enter only at the output; everything else is passed along the wires.

🍞 THINK OF IT LIKE... TRACING A BAD CAKE BACK TO ITS SUPPLIERS

A customer sends a cake back: too sweet by a certain amount (the loss, and its slope $p - y$). The baker used flour from one supplier and sugar from another, in known proportions (the wires v). The sugar supplier's share of the blame is: how wrong the cake was × how much of it was sugar × how sensitive that supplier's sugar is to its own settings (the slope g'). Nobody phones the customer to ask about the sugar supplier; the blame is *computed* from the baker's blame and the recipe. And a supplier whose ingredient was left out of this cake (a ReLU neuron that was off) gets no blame today — not because it is innocent, but because a small change in it would have changed nothing.

💡 THE BACKWARD ALGORITHM, FOR ANY DEPTH

Seed	$D^L = \frac{1}{n}(P - Y)$ at the output layer L .
For each layer $l = L$ down to 1	$\nabla_{W^l} L = A^{(l-1)T} D^l$ and $\nabla_{b^l} L = 1^T D^l$ (column sums). Then, if $l > 1$, pass the blame back: $D^{l-1} = (D^l W^{lT}) \odot g'(Z^{l-1})$.
Output	a gradient for every knob, of the same shape as the knob. Cost: two matrix products per layer — about the same as the forward pass.

The forward pass reads the network left to right and remembers every Z and A . The backward pass reads it right to left and spends them. Together they are a *reverse-mode* way of taking derivatives, and they cost about the same whether the network has a hundred knobs or a hundred million.

One backward pass by hand, every number

A network small enough to finish on paper: two inputs, two ReLU hidden neurons, one sigmoid output. W^1 is the identity (each hidden neuron just copies one input), $b^1 = 0$, $W^2 = (1, -1)^T$, $b^2 = 0$. One example, $x = (1, 1)$, truth $y = 0$ — so $n = 1$ and there is no division. First the plain way, neuron by neuron; then the same numbers as matrices.

💡 PLAIN MATHS — forward, then blame, one neuron at a time

F	$z_1 = 1 \cdot 1 + 0 \cdot 1 + 0 = 1, \quad z_2 = 0 \cdot 1 + 1 \cdot 1 + 0 = 1, \quad a_1 = a_2 = 1,$	Forward. Loss $L = -\log(1 - 0.5) = 0.693$.
B1	$\delta^{(2)} = p - y = 0.5 - 0 = 0.5$	Output blame — link 4.
B2	$\frac{\partial L}{\partial v_1} = \delta^{(2)} a_1 = 0.5, \quad \frac{\partial L}{\partial v_2} = \delta^{(2)} a_2 = 0.5, \quad \frac{\partial L}{\partial c} = 0.5$	Output knobs: blame × input (the hidden outputs).
B3	$\delta_1 = \delta^{(2)} v_1 \text{relu}'(z_1) = 0.5 \cdot 1 \cdot 1 = 0.5, \quad \delta_2 = \delta^{(2)} v_2 \text{relu}'(z_2) = 0.5 \cdot 1 \cdot 1 = 0.5$	Hidden blames — step 7: blame of the output × the wire × the slope (both ReLUs are on, so slope 1).
B4	$\frac{\partial L}{\partial w_{11}} = \delta_1 x_1 = 0.5, \quad \frac{\partial L}{\partial w_{21}} = \delta_1 x_2 = 0.5, \quad \frac{\partial L}{\partial w_{12}} = \delta_2 x_1 = -0.5, \quad \frac{\partial L}{\partial w_{22}} = \delta_2 x_2 = -0.5$	Hidden knobs: blame × input, for all six.

↓ the same eight numbers, as **four matrix lines**

💡 MATRIX FORM — $n = 1$: $\mathbf{X}$ is one row

$$\mathbf{Z}^1 = [1 \ 1] \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + [0 \ 0] = [1 \ 1], \quad \mathbf{A}^1 = \text{relu}(\mathbf{Z}^1) = [1 \ 1], \quad \mathbf{Z}^2 = [1 \ 1] \begin{bmatrix} 1 \\ -1 \end{bmatrix} + 0 = 0, \quad \mathbf{P} = \sigma(0) = 0.5$$

$$\mathbf{D}^2 = \mathbf{P} - \mathbf{Y} = 0.5 \quad \nabla_{\mathbf{W}^2} L = \mathbf{A}^{1\top} \mathbf{D}^2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} 0.5 = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \quad \nabla_{b^2} L = 0.5$$

$$\mathbf{D}^1 = (\mathbf{D}^2 \mathbf{W}^{2\top}) \odot \text{relu}'(\mathbf{Z}^1) = (0.5 \begin{bmatrix} 1 & -1 \end{bmatrix}) \odot [1 \ 1] = [0.5 \ -0.5]$$

$$\nabla_{\mathbf{W}^1} L = \mathbf{X}^\top \mathbf{D}^1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0.5 & -0.5 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & -0.5 \end{bmatrix} \quad \nabla_{b^1} L = [0.5 \ -0.5]$$

the same numbers as B1–B4, in the same places: column j of $\nabla_{\mathbf{W}^1} L$ is neuron j 's blame times the inputs

Read the blames. The truth is 0 and the network said 0.5, so the output score must fall: $\delta^{(2)}$ is positive. Hidden neuron 1 pushes the score *up* (its wire is +1), so it receives positive blame — turn it down. Hidden neuron 2 pushes the score *down* (its wire is -1), so its blame is negative — turn it up. Every knob feeding neuron 1 will shrink, every knob feeding neuron 2 will grow, and the output wires will move to lean less on neuron 1 and more on neuron 2. Nobody told the hidden neurons the label. The wires did. The next chapter takes the step.

💡 STOP AND THINK

Same network, but suppose the forward pass had given $\mathbf{Z}^1 = (1, -1)$. What is $\mathbf{D}^1$ now? What happens to the knobs of hidden neuron 2 on this step?

✓ OUR ANSWER

$\text{relu}'(-1) = 0$, so $\mathbf{D}^1 = (0.5 \begin{bmatrix} 1 & -1 \end{bmatrix}) \odot [1 \ 0] = (0.5, 0)$. Neuron 2 receives no blame, and none of its knobs move — from this example. Not because it is innocent, but because at this moment a small nudge to its knobs would change nothing at the output. Gradients are *local*. A ReLU neuron that is off for every example never learns again; people call it a **dead** neuron (Chapter 6).

✨ SPARK — TRUST, BUT VERIFY

How do you know a page of formulas is right? Test it against the definition of a slope. Nudge one knob up by a tiny ϵ , run the forward pass, note the loss; nudge it down by ϵ , note the loss; the slope is $(L_+ - L_-) / 2\epsilon$. Do this for every knob and compare with what backpropagation produced. The notebook does exactly this for every weight of a 2-3-1 tanh network, with $\epsilon = 10^{-6}$: the largest disagreement is about 10^{-10} . This **gradient check** is the standard way to debug a hand-written backward pass. It also shows why backpropagation matters: nudging needs two forward passes *per knob*; backpropagation finds every gradient in one backward pass. For a million knobs that is the difference between a second and a month.

The method — reverse-mode differentiation — was published by Seppo Linnainmaa in 1970 and applied to neural networks by Paul Werbos in 1974; it became famous in 1986 when Rumelhart, Hinton and Williams showed hidden layers learning useful features with it. Today every framework's `loss.backward()` is this chapter, automated.

📖 WORDS TO KNOW

Backpropagation — computing every neuron's blame from the output backwards, then every gradient as blame $\times$ input. **Blame δ^l (delta), $\mathbf{D}^l$ for a batch** — $\partial L / \partial \mathbf{Z}^l$: how the loss changes if a neuron's weighted sum nudges. **Credit assignment** — the problem of deciding how much each hidden knob contributed to the error; backpropagation solves it. **$\mathbf{W}^\top$** — the transpose: rows and columns swapped; blame goes back through it. **Local gradient** — a slope at the current knobs only. **Gradient check** — comparing backpropagation with nudging each knob.

6

The update, and XOR learned from nothing

Subtract the gradient, times the learning rate, from every knob at once. Then start from random numbers and watch the puzzle of Book 2 dissolve — and learn why it sometimes does not.

Finish the hand example

Chapter 5 left us with a gradient for every knob. The update is Book 3's, applied to all of them together: each knob moves *against* its gradient by η times its size — $w \leftarrow w - \eta \partial L / \partial w$ for every w . With $\eta = 1$ (to keep the arithmetic clean):

🔔 **PLAIN MATHS** — every knob, minus its gradient

- 1 $w_{11} \leftarrow 1 - 0.5 = 0.5, \quad w_{21} \leftarrow 0 - 0.5 = -0.5, \quad w_{12} \leftarrow 0 - (-0.5) = 0.5$ **hidden weights:** each moves against its own gradient from Chapter 5, step B4
- 2 $b_1 \leftarrow 0 - 0.5 = -0.5, \quad b_2 \leftarrow 0 - (-0.5) = 0.5, \quad v_1 \leftarrow 1 - 0.5 = 0.5$ **biases and output knobs**
- 3 $z_1 = 0.5 - 0.5 - 0.5 = -0.5, \quad z_2 = 0.5 + 1.5 + 0.5 = 2.5, \quad a_1 = 0, \quad a$ **forward again** on the same $\mathbf{x} = (1, 1)$
- 4 $p = \sigma(-4.25) = 0.014, \quad L = -\log(1 - 0.014) = 0.014$ (was 0.693) **the loss after one step**

↓ the same update as **four matrix subtractions** — each gradient has the shape of its knob

🔔 **MATRIX FORM** — $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} L$, layer by layer

$$\begin{aligned} \mathbf{W}^1 &\leftarrow \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 1 \cdot \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & -0.5 \end{bmatrix} = \begin{bmatrix} 0.5 & 0.5 \\ -0.5 & 1.5 \end{bmatrix} & \mathbf{b}^1 &\leftarrow \begin{bmatrix} 0 & 0 \end{bmatrix} - \begin{bmatrix} 0.5 & -0.5 \end{bmatrix} = \begin{bmatrix} -0.5 & 0.5 \end{bmatrix} \\ \mathbf{W}^2 &\leftarrow \begin{bmatrix} 1 \\ -1 \end{bmatrix} - \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 \\ -1.5 \end{bmatrix} & b^2 &\leftarrow 0 - 0.5 = -0.5 \\ \mathbf{Z}^1 &= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 0.5 & 0.5 \\ -0.5 & 1.5 \end{bmatrix} + \begin{bmatrix} -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} -0.5 & 2.5 \end{bmatrix}, & \mathbf{A}^1 &= \begin{bmatrix} 0 & 2.5 \end{bmatrix}, & \mathbf{Z}^2 &= \begin{bmatrix} 0 & 2.5 \end{bmatrix} \begin{bmatrix} 0.5 \\ -1.5 \end{bmatrix} - 0.5 = -4.25, & \mathbf{P} &= 0.014 \end{aligned}$$

Every change pushed the same way. Hidden neuron 1, which was helping the output say “yes”, has been switched off for this input (z_1 went from 1 to -0.5). Hidden neuron 2, whose wire pulls the output down, was turned up ($1 \rightarrow 2.5$), and its wire was pulled harder ($-1 \rightarrow -1.5$). The bias b^2 dropped. One step took the loss from 0.693 to 0.014.

⚠️ **CAREFUL — THIS STEP WAS TOO GOOD TO BE TYPICAL**

One example, and a learning rate of 1, is a caricature. Real training averages the blame over a batch of examples that pull in different directions, and uses a smaller η , so each step is a small nudge. The point of the hand example is the *mechanics*: forward, blame, gradient, subtract — every number accounted for. Also notice that the network is now very sure that $(1, 1)$ is a “no”, having seen only that one example; it would say “no” to $(0, 1)$ as well, with $p = 0.03$. Learning from one example over-learns it. That is why we cycle through all of them, many times.

🔔 **THE TRAINING LOOP, COMPLETE**

Start	every $\mathbf{W}^l$ small and <i>random</i> ; every $\mathbf{b}^l$ zero.
Repeat for E epochs	shuffle the training set; for each mini-batch (or the whole set): forward (keep every $\mathbf{Z}^l, \mathbf{A}^l$) → loss → backward ($\mathbf{D}^L \dots \mathbf{D}^1$, every $\nabla_{\mathbf{W}^l} L$ and $\nabla_{\mathbf{b}^l} L$) → update : $\mathbf{W}^l \leftarrow \mathbf{W}^l - \eta \nabla_{\mathbf{W}^l} L, \mathbf{b}^l \leftarrow \mathbf{b}^l - \eta \nabla_{\mathbf{b}^l} L$, for every layer.
Watch	the training loss per epoch, and the accuracy on examples the network has never seen.

This is Book 1's loop — guess, measure, adjust — with the “adjust” step now reaching every knob in every layer. Backpropagation is not the learning rule; gradient descent is. Backpropagation just delivers the gradient.

Why the start must be random

🔔 **STOP AND THINK**

Books 3 and 4 started every knob at zero and it worked fine. Suppose we start the XOR network with every weight at zero (tanh hidden layer). Trace one step: what is $\mathbf{A}^1$? What is $\mathbf{D}^1$? Which knobs move?

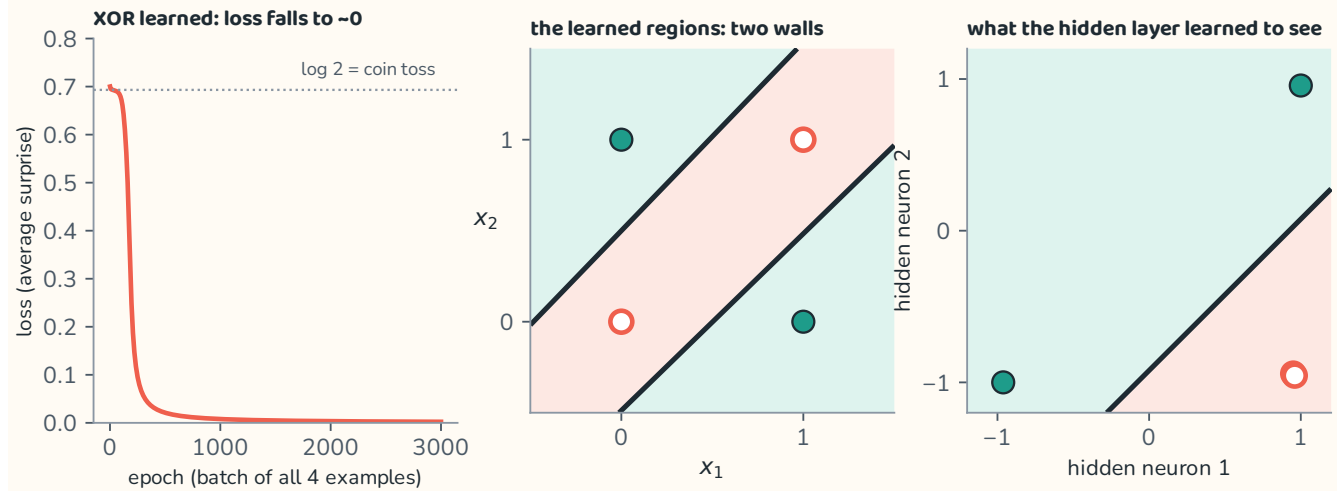
✓ **OUR ANSWER**

$\mathbf{Z}^1 = 0$, so $\mathbf{A}^1 = \tanh(0) = 0$. Then $\nabla_{\mathbf{W}^2} L = \mathbf{A}^{1T} \mathbf{D}^2 = 0$, and $\mathbf{D}^1 = (\mathbf{D}^2 \mathbf{W}^{2T}) \odot g'(\mathbf{Z}^1) = 0$ because $\mathbf{W}^2 = 0$. Only b^2 moves. Next step, the same. The network never leaves zero. Even if every weight starts at the same *non-zero* value, the two hidden neurons see the same inputs with the same weights, produce the same output, receive the same blame and make the same update — forever identical. Two copies of one neuron are one neuron. A random start **breaks the symmetry**: each hidden neuron begins slightly different, so the blame reaches each differently, and they specialise.

How random? Too small and every z is near zero, where tanh and sigmoid look like straight lines and nothing bends; too large and z is huge, where tanh and sigmoid are flat and their slope — one of the links in the chain — is nearly zero, so the blame fades on its way back. The usual recipe scales each weight by $1/\sqrt{\text{inputs to that neuron}}$, or $\sqrt{2/\text{inputs}}$ for ReLU (the Xavier and He initialisations). Our code uses the second.

XOR, learned from nothing

The setup: 2 inputs, 2 tanh hidden neurons, 1 sigmoid output, the surprise loss, all four examples in every batch, $\eta = 0.5$, random seed 0, 3000 epochs. No knob is set by hand.



Left: the loss sits at log 2 (a coin toss) for the first hundred-odd epochs, then collapses. **Middle:** the learned decision regions — a band between two walls, “yes” inside. **Right:** the four inputs as the hidden layer now sees them: the two “yes” cases pushed apart from the two “no” cases so that one line separates them — the picture from Chapter 1, found by the network itself.

epoch	1	100	300	500	1000	3000
loss	0.700	0.676	0.058	0.021	0.008	0.002

The knobs it found, rounded, and what the two hidden neurons turned into:

$$\mathbf{W}^1 = \begin{bmatrix} -3.81 & -3.80 \\ 3.92 & 3.67 \end{bmatrix} \quad \mathbf{b}^1 = [1.83 \quad -1.76] \quad \mathbf{W}^2 = \begin{bmatrix} -6.61 \\ 6.63 \end{bmatrix} \quad b^2 = 6.10$$

the first column of $\mathbf{W}^1$ is neuron 1's two weights, the second column neuron 2's — read the plain versions below straight off the columns

$$\begin{aligned} h_1 &= \tanh(-3.81 x_1 + 3.92 x_2 + 1.83) \rightarrow \begin{matrix} (0,0) & (0,1) & (1,0) & (1,1) \\ +0.95 & +1.00 & -0.96 & +0.96 \end{matrix} \quad \text{“not (1,0)”} \\ h_2 &= \tanh(-3.80 x_1 + 3.67 x_2 - 1.76) \rightarrow \begin{matrix} -0.94 & +0.96 & -1.00 & -0.96 \end{matrix} \quad \text{“is it (0,1)?”} \\ z^{(2)} &= -6.61 h_1 + 6.63 h_2 + 6.10 \rightarrow p = \begin{matrix} 0.002 & 0.997 & 0.997 & 0.001 \end{matrix} \quad \text{truth } 0 \ 1 \ 1 \ 0 \checkmark \end{aligned}$$

Look at what it invented. Neuron 2 became a detector for the case (0, 1). Neuron 1 became a detector for “anything except (1, 0)”. The output says yes when h_2 is high or h_1 is low — that is, for (0, 1) or (1, 0). It is not the “count the inputs” trick we designed in Chapter 1, and not the OR-NAND-AND circuit of Book 2. It is a third solution, and nobody chose it. **The hidden layer learned its own features from four examples and a loss.** That is the sentence Book 1 promised.

💡 THINK OF IT LIKE... THE FLAT STRETCH BEFORE THE DROP

Why the long plateau at log 2? At the start the knobs are small, so both hidden neurons output nearly zero, and the output neuron has nothing to lean on: whatever it does, p stays near 0.5. The blame reaching the hidden layer is small too — it passes through the small $\mathbf{W}^2$. Everyone is waiting for everyone else. Then a tiny asymmetry grows: one hidden neuron becomes slightly useful, the output leans on it, which sends it more blame, which makes it more useful. The loss falls off a cliff. Learning curves are often flat-then-sudden for exactly this reason; a flat curve does not always mean nothing is happening.

Luck, and getting stuck

Change nothing except the random start and run twenty times. The notebook counts how many runs reach a loss near zero within 3000 epochs:

Network	Hidden activation	η	Solved XOR	Where the others got stuck
2-2-1	tanh	0.5	9 / 20	loss 0.35: sure about two cases, exactly 0.5 on the other two
2-2-1	ReLU	0.5	2 / 20	loss 0.35, 0.48 or 0.69: one or both hidden neurons dead
2-2-1	sigmoid	2.0	17 / 20	loss 0.35, as for tanh
2-4-1	tanh	0.5	20 / 20	—
2-4-1	ReLU	0.5	15 / 20	loss 0.35 or 0.48

Two lessons hide in this table. First, **the smallest network that can solve a problem is usually the hardest to train**. Two hidden neurons must both land on useful features; four give the search some slack, and the tanh 2-4-1 network never failed. Second, the failures are not random noise — each is a recognisable trap:

The collapsed pair (loss 0.35). In the stuck tanh runs, the network is certain about (0, 0) and (1, 0) — and outputs exactly 0.5 for (0, 1) and (1, 1), because its hidden layer maps those two inputs to the same point. One has truth 1, the other 0; their blames are equal and opposite and cancel; nothing moves. The hidden layer did the opposite of its job, merging two inputs that needed separating, and once merged there is no gradient to unmerge them. This is a **local minimum**: every small change makes things worse, though a large one would help.

The dead neuron (loss 0.48, 0.69). In the stuck ReLU runs, a hidden neuron is off — Z^1 negative — for every one of the four inputs. Its slope is 0 everywhere it is used, so by Chapter 5 it receives no blame, so its knobs never move, so it stays off. At loss 0.69 both neurons are dead and the network outputs 0.5 for everything. A dead ReLU is a real hazard in big networks too; small learning rates, good initialisation, and variants such as “leaky” ReLU (a small slope instead of 0 when off) are the usual cures.

The learning rate, once more

Same 2-2-1 tanh network, seed 0, three learning rates. With $\eta = 0.05$ the plateau lasts about a thousand epochs and the loss is still 0.057 at epoch 3000 — ten times slower, as you would expect. With $\eta = 0.5$ it is done by epoch 500. With $\eta = 5$ the loss first *jumps up* to 1.39 as the knobs overshoot, then happens to land and finishes fastest of all. On this four-example toy a wild step got lucky; on real data with real noise it usually tips the loss into “nan”. Start around 0.01–0.1, look at the curve, and adjust. Mini-batches (Book 4) apply unchanged: the moons in the next chapter are trained sixteen points at a time.

TRY IT YOURSELF — A SECOND STEP

Take the updated knobs from the top of this chapter and feed the example $\mathbf{x} = (0, 1)$, truth $y = 1$. Run the forward pass and find p and the loss. Then find $\delta^{(2)}$, $\nabla_{\mathbf{W}^2} L$, $\mathbf{D}^1$ and $\nabla_{\mathbf{W}^1} L$ — and say in words what the step will do to hidden neuron 1. Check upside down:

$Z^1 = (-1, 2)$, $A^1 = (0, 2)$, $Z^2 = (-3.5, -0.971)$, $A^2 = (0, 1.941)$, $D^1 = (0, 1.46)$, $\nabla_{\mathbf{W}^2} L$ has only one non-zero entry, 1.46, in row 1, column 2. Neuron 1 is off for this input, receives no blame, and does not move at all.

WORDS TO KNOW

Update — $\mathbf{W}^l \leftarrow \mathbf{W}^l - \eta \nabla_{\mathbf{W}^l} L$ for every layer at once.

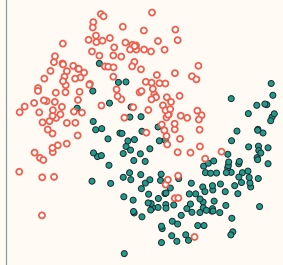
Initialisation — the random starting knobs; scaled by $1/\sqrt{\text{inputs}}$, or $\sqrt{2/\text{inputs}}$ for ReLU. **Symmetry breaking** — why they must be random: identical neurons stay identical. **Plateau** — a flat stretch of the loss curve while small knobs wait for each other. **Local minimum** — a place where every small change makes the loss worse. **Dead ReLU** — a neuron that is off for every example and so never receives blame. **Seed** — the number that fixes the random start, so a run can be repeated.

Depth and width

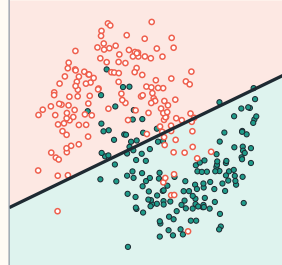
More neurons in a row, or more rows? Two crescent moons, eight networks, and an honest reading of what the numbers do and do not say.

XOR has four points, and a network either gets all of them or it does not. To see what size buys, we need a problem with a shape. Here is one: 300 points in two interleaving crescents, with some noise so that a few points of each colour stray into the other moon. We hide 90 of them (30%) and train on the remaining 210 — the referee from Books 3 and 4.

300 points, two moons



one sigmoid neuron (Book 4): a straight wall, 86%

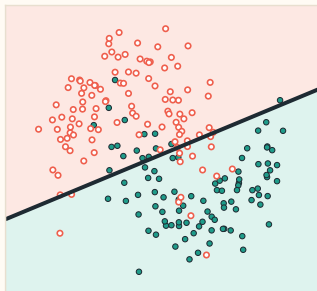


Two moons, and the best a straight wall can do. Book 4's single sigmoid neuron reaches 86% on the hidden points: the two tips of the crescents are on the wrong side of any line. Everything below asks what hidden layers add.

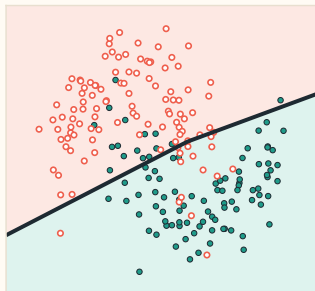
Eight networks, one picture

All eight use ReLU hidden layers, a sigmoid output, mini-batches of 16, $\eta = 0.05$, 400 epochs and the same random seed. Only the shape changes: the first row varies the **width** of a single hidden layer; the second row adds **depth**.

width 1
train 85% · hidden test 87%



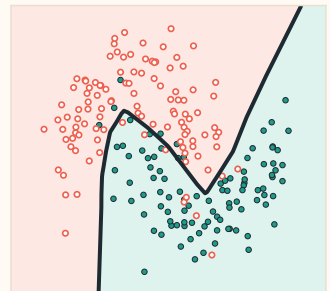
width 2
train 85% · hidden test 86%



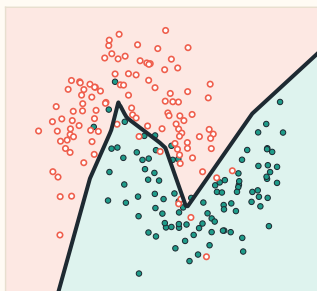
width 4
train 92% · hidden test 90%



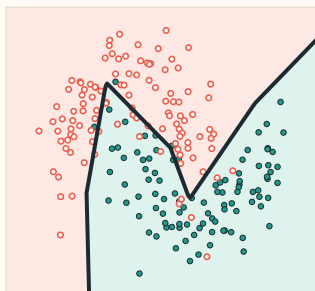
width 32
train 92% · hidden test 93%



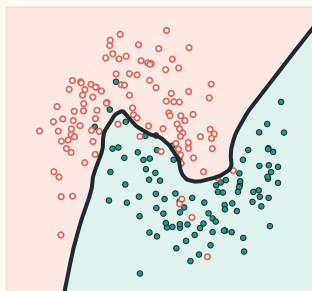
depth 2 (4 + 4)
train 92% · hidden test 94%



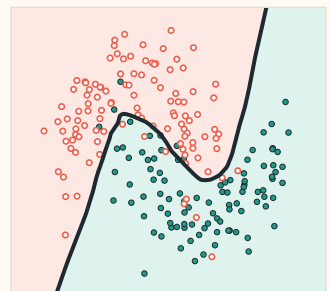
depth 3 (4 + 4 + 4)
train 90% · hidden test 96%



depth 2, width 64
train 93% · hidden test 93%



width 256
train 93% · hidden test 94%



The boundary each network drew, with its accuracy on the 210 training points and on the 90 hidden ones. Width 1 and 2 are still a wall. Width 4 has corners. Width 32 has so many small corners that it looks like a curve. In the second row, two or three layers of only four neurons trace the crescents at least as well as one layer of 256.

Network (inputs-hidden...-output)	Hidden layers	Knobs	Training	Hidden test	What the boundary looks like
2-1-1	1	5	85%	87%	a straight wall — one hinge cannot bend a boundary
2-2-1	1	9	85%	86%	still a wall; the second hinge found nothing useful
2-4-1	1	17	92%	90%	three sharp corners; the tips are now on the right side
2-32-1	1	129	92%	93%	many small corners; reads as a curve
2-4-4-1	2	37	92%	94%	a bent band that follows the crescents
2-4-4-4-1	3	57	90%	96%	the best hidden-test score, with 57 knobs
2-64-64-1	2	4,417	93%	93%	smooth; about as good as width 32
2-256-1	1	1,025	93%	94%	smooth; about as good as depth 2

Knobs = every weight and bias: for 2-4-4-1, $(2 \cdot 4 + 4) + (4 \cdot 4 + 4) + (4 \cdot 1 + 1) = 37$. The straight wall of Book 4 has 3.

Reading the table honestly

The hidden set has 90 points, so one point is 1.1%. Run the same experiment with a different random seed and the numbers shuffle by a point or two: on our second run, depth 3 scored 94% and width 256 scored 96%. So the *ranking* among the good networks is noise. Three things are not noise, and they are the lessons:

Below about four hinges, nothing happens. Width 1 and width 2 cannot beat the straight wall (86%), because a boundary needs several folds before it can bend around a crescent. Once there are enough hinges — from 17 knobs upwards — every network lands in the same 90–96% band.

Depth is a cheap way to get hinges. Three layers of four neurons (57 knobs) draw a boundary at least as good as one layer of 256 neurons (1,025 knobs). A fold made in the first layer is reused by every neuron in the second; the corners *compound*. Width adds folds side by side; depth folds the already-folded sheet.

There is a ceiling, and it is the data. Some points sit deep inside the wrong moon. No boundary can collect them without wrapping around individual points — memorising — and a network that did so would score better on training and worse on the hidden set. The noise sets the ceiling; no architecture lifts it.

💡 BIG IDEA — CAPACITY, AND WHO CHECKS IT

The number and arrangement of a network's knobs is its **capacity**: how complicated a boundary it *could* draw. Too little capacity and it cannot follow the shape of the data (width 1 and 2 above — **underfitting**). Too much, trained too long on too few examples, and it follows the noise (**overfitting**). You cannot tell which is happening from the training score; you can only tell from examples the network has never seen. That is why every book in this series hid some of the data before training. The hidden set is the referee, and it has the last word on every architecture choice.

💡 STOP AND THINK

The 2-256-1 network has 1,025 knobs and only 210 training points. Shouldn't it be able to memorise every training point and score 100% on them? It scored 93%. Why?

✓ OUR ANSWER

It could, given many more epochs and a bigger step. But gradient descent from small random knobs is lazy in a useful way: the first thing it finds is a simple boundary that gets most points right, and only much later does it start adding wiggles to collect the stragglers. After 400 small steps it has drawn a smooth band and stopped bothering. This "simple first" habit is one reason enormous networks work at all — and the reason people watch the hidden-test curve and *stop early* when it turns down. The danger is real, though: train that same network on only 40 points for 3,000 epochs and it reaches 84% on the hidden set — the same as a 4-neuron network trained the same way. Too few examples hurt more than any choice of size helps.

∞ THINK OF IT LIKE... FOLDING PAPER, AGAIN

Chapter 2 said each hidden neuron folds the sheet and the output makes one straight cut. Width is the number of folds you make in one go. Depth is folding the *already folded* sheet: each new fold goes through every layer of paper at once, so the number of separate pieces after the cut grows much faster. Four folds, then four more, then four more, cut more pieces than 256 folds made side by side. That is the whole argument for depth — and the reason the field is called *deep* learning rather than *wide* learning.

⚠ CAREFUL — DEPTH HAS A PRICE, AND RELU PAYS IT

Blame going back through a layer is multiplied by that layer's slope (Chapter 5). The sigmoid's slope is at most 0.25, so after ten sigmoid layers the blame reaching the first layer is at most 0.25^{10} — about one millionth — of what left the output. The early layers learn nothing: the **vanishing gradient** problem, which kept networks shallow for years. ReLU's slope is exactly 1 when the neuron is on, so blame passes through undiminished. That, more than anything, is why the hidden layers in this chapter use ReLU and why deep networks became possible after about 2010. (The notebook's "Things to try" builds a six-layer sigmoid network and prints the first layer's gradient. Look at it.)

✦ SPARK — HOW TO CHOOSE A SIZE, IN PRACTICE

Nobody derives the width and depth; they are chosen by experiment, guided by three habits. Start small: one or two hidden layers of 16 to 64 ReLU neurons handles most tabular problems. Watch two curves, training loss and hidden-test accuracy, every few epochs. If both are poor, the network is too small or has not trained long enough — add neurons, a layer, or epochs. If training is excellent and the hidden test is poor, it is memorising — add data, stop earlier, or add a *regulariser* (Chapter 8). And change one thing at a time, because with four random starts and three learning rates you can prove anything.

🔧 TRY IT YOURSELF

In the notebook, change the moons' noise from 0.22 to 0.35 and re-run the eight networks. Predict first: which numbers will move — the training scores, the hidden-test scores, or the gap between them — and in which direction? Then try `n_samples=60` with the noise back at 0.22. Which matters more here, points or noise?

📖 WORDS TO KNOW

Width — neurons in a layer. **Depth** — number of hidden layers.
Capacity — how complicated a boundary the knobs could draw.
Underfitting / overfitting — too simple for the data / following its noise. **Generalisation** — doing well on examples never seen. **Early stopping** — stop when the hidden-test curve turns down.
Vanishing gradient — blame fading through many small slopes.

Nova, at last — and what lies beyond

The wake word as a network, the road ahead in one paragraph per turn, and the forty lines of code that ran every experiment in this book.

STORY

Asha puts the five notebooks side by side. Book 1 ended with a promise: sixteen thousand numbers in, yes or no out, and a stack of layers that learn their own features. She has now built every piece of that stack. “Nova,” she says, “this time it is your turn.” She plugs in the microphone.

Nova's brain, assembled

Engineers rarely feed raw sound to a network. First they turn one second of sound into a picture: a hundred thin time-slices, each recording how loud the sound is in each of forty pitch bands — about 4,000 numbers instead of 16,000, arranged so that “Hey” and “Nova” look like shapes. Those 4,000 numbers are X . After that, nothing new happens:

Piece	What it is for Nova	Built in
Examples	thousands of clips of “Hey Nova”, labelled 1, and many more of everything else, labelled 0; some held back as the referee	Book 1
The neuron	weighted sum + bias, then a bend	Book 2
The loss and the walk	a score for how wrong a guess is; step every knob downhill by its gradient	Book 3
The output	a sigmoid giving $p(\text{“Hey Nova”})$, the surprise loss, blame $p - y$, mini-batch SGD, and a threshold chosen for the cost of a false alarm versus a miss	Book 4
The layers	4000 inputs $\rightarrow$ 256 ReLU $\rightarrow$ 64 ReLU $\rightarrow$ 1 sigmoid; forward as three matrix products; backpropagation through the same wires; random start	Book 5

Count the knobs: $4000 \cdot 256 + 256$ in the first layer, $256 \cdot 64 + 64$ in the second, $64 + 1$ at the output — **1,040,769** in all. Every one of them is trained by the same four lines: forward, blame, gradient, subtract. What the first layer’s neurons learn to see is not chosen by anyone: after training, one responds to a hiss followed by a burst (the start of “Hey”), another to a rising pitch, another to silence-then-sound. The second layer combines them into “sounds like Hey”, “sounds like No-va”. The output weighs those. Book 1 called this learning the features; Chapter 6 showed it on four points. It is the same thing at a million knobs.

SPARK — WHAT LIES BEYOND, ONE PARAGRAPH EACH

Convolutional networks. In a picture, an edge is an edge wherever it appears. A convolutional layer is a small neuron — say 3×3 inputs — that slides across the image and reuses the same nine knobs at every position. Far fewer knobs, and the network cannot help learning features that work everywhere. Stack a few dozen such layers and you have the networks that see.

Recurrent networks and transformers. A sentence or a sound is a sequence, and its length varies. Recurrent networks read one step at a time, carrying a hidden state along. Transformers instead let every position *attend* to every other — a learned weighting that says which parts of the input to listen to for each output. The language models you talk to are transformers: neurons, softmax, cross-entropy, backpropagation, at a scale of billions of knobs.

Regularisation. Chapter 7 met overfitting. The cures: *weight decay* (add a small penalty for large knobs to the loss, so the gradient nudges them toward zero), *dropout* (randomly switch off a fraction of the hidden neurons on each step, so no neuron can rely on another), *early stopping*, and *data augmentation* (make more examples by shifting, adding noise, changing speed — for Nova, the same clip at three volumes is three clips).

Better walkers. Plain gradient descent takes the same kind of step for every knob. *Momentum* keeps a running average of recent gradients, so the walk does not zig-zag. *Adam* also scales each knob’s step by how big its gradients have recently been, so a rarely-blamed knob still moves. They are all the same loop with a smarter “subtract”.

Frameworks. In PyTorch or JAX you write only the forward pass. The framework records every operation and runs Chapter 5 for you — `loss.backward()` is backpropagation, automated for any graph of operations, and a GPU does Chapter 3’s matrix products thousands at a time. You now know exactly what those two lines are doing, which is more than most people who call them.

CAREFUL — THIS NOVA WORKS, BUT IT IS NOT THE BEST NOVA

A plain MLP on a sound picture does wake up for “Hey Nova” and stay asleep for most other things, given enough clips. The wake-word detectors in real devices use two of the refinements above: *convolutional* layers, which look for the same small pattern anywhere in the sound picture, and *recurrent* or *attention* layers, which handle sound as a sequence. Both are made of the neurons, losses and backpropagation in these five books; they just wire the neurons more cleverly. Nothing new in kind is needed — only new arrangements.

The network, in forty lines

This is the whole of Books 2 to 5 as code: any number of layers, any of the three hidden activations, a sigmoid output with the surprise loss, full-batch or mini-batch training. Read backward against Chapter 5 — it is the three-line algorithm, repeated layer by layer, going backwards.

```
class MLP:
    def __init__(self, sizes, hidden='relu', lr=0.1, seed=0):
        # sizes = [inputs, hidden..., outputs]
        rng = np.random.default_rng(seed); self.W, self.b = [], []
        for n_in, n_out in zip(sizes[:-1], sizes[1:]):
            self.W.append(rng.normal(0, 1, (n_in, n_out)) * np.sqrt(2/n_in)) # small RANDOM start (Chapter 6)
            self.b.append(np.zeros(n_out))
        self.g, self.g_d = ACT[hidden]; self.lr = lr; self.loss_hist = [] # the activation and its slope

    def forward(self, X):
        # Chapter 3
        A = X; self.Zs, self.As = [], [X]; L = len(self.W)
        for l in range(L):
            Z = A @ self.W[l] + self.b[l] # Z = A W + b, whole batch at once
            A = self.g(Z) if l < L-1 else sigmoid(Z) # hidden: bend; output: squash
            self.Zs.append(Z); self.As.append(A) # keep them for the backward pass
        return A # P: one probability per row

    def backward(self, Y):
        # Chapter 5
        n = Y.shape[0]; L = len(self.W); gW, gb = [None]*L, [None]*L
        D = (self.As[-1] - Y) / n # the seed: output blame (P - Y)/n
        for l in reversed(range(L)):
            gW[l] = self.As[l].T @ D # gradient = (input seen)^T * blame
            gb[l] = D.sum(axis=0) # bias gradient = column sums
            if l > 0: D = (D @ self.W[l].T) * self.g_d(self.Zs[l-1]) # blame back through W^l, then * slope
        return gW, gb

    def step(self, gW, gb):
        # Chapter 6
        for l in range(len(self.W)):
            self.W[l] -= self.lr * gW[l]; self.b[l] -= self.lr * gb[l] # every knob, against its gradient

    def fit(self, X, Y, epochs=1000, batch=None, seed=0):
        rng = np.random.default_rng(seed); n = len(X)
        for _ in range(epochs):
            idx = rng.permutation(n); size = batch or n # shuffle; whole set or mini-batches
            for s in range(0, n, size):
                bi = idx[s:s+size]; self.forward(X[bi]); self.step(*self.backward(Y[bi]))
            self.loss_hist.append(bce(self.forward(X), Y)) # watch the loss per epoch
        return self

    def predict(self, X): return (self.forward(X) >= 0.5).astype(int)
```

Usage, from the notebook: `MLP([2, 2, 1], hidden='tanh', lr=0.5).fit(X, Y, epochs=3000)` learns XOR; `MLP([2, 4, 4, 4, 1]).fit(Xs, Y, epochs=400, batch=16)` is the 96% network of Chapter 7. Change sizes and nothing else changes — that is what “vectorised” bought us. **Notebook map:** §1 the hand-built forward pass, every number (Ch. 1, 3) · §2 the class above (Ch. 3, 5, 6) · §3 the gradient check (Ch. 5) · §4 one step by hand (Ch. 5–6) · §5 XOR learned and §5b twenty random starts (Ch. 6) · §6 depth and width and §6b forty points (Ch. 7) · §7 things to try: zero starts, a stuck run, vanishing gradients, overfitting on purpose, a softmax output.

Look back: five books, one idea

The whole series in eight sentences, a self-check, questions worth carrying, and the end that is a beginning.

The story in eight sentences

1. **Book 1.** When you cannot write the rule, collect examples and build a machine that finds it: a model with knobs, a loss that scores a guess, a walk that turns the knobs downhill, and a hidden set that referees.
2. **Book 2.** One neuron is a weighted sum plus a bend; the perceptron rule learns any straight wall, and fails at XOR because for XOR no straight wall exists — and nobody could tell a hidden neuron how wrong it was.
3. **Book 3.** Make the neuron and the loss smooth and the *slope* says which way to turn each knob: gradient = error × input, step by η , and the chain rule multiplies the links.
4. **Book 4.** Sigmoid plus surprise gives $p - y$ at the output; step one example or a mini-batch at a time; softmax and one-hot handle many classes with the same $P - Y$.
5. **This book, Chapters 1–2.** A hidden layer re-draws the map until a straight wall is enough — but only if it *bends*; stacked straight lines collapse into one straight line, so every hidden neuron needs an activation.
6. **Chapters 3–4.** Forward propagation is $z_j = \sum_i w_{ij}a_i + b_j$, $a_j = g(z_j)$ for each neuron — as matrices $\mathbf{Z}^l = \mathbf{A}^{l-1}\mathbf{W}^l + \mathbf{b}^l$, $\mathbf{A}^l = g(\mathbf{Z}^l)$, layer after layer, the whole batch at once; the output's blame $(\mathbf{P} - \mathbf{Y})/n$ is the seed.
7. **Chapters 5–6.** Backpropagation passes the seed back through each $\mathbf{W}^T$ and multiplies by each slope — $\delta_j = (\sum_m \delta_m^{(2)} w_{jm})g'(z_j)$, or $\mathbf{D}^{l-1} = (\mathbf{D}^l \mathbf{W}^{lT}) \odot g'(\mathbf{Z}^{l-1})$; every gradient is (the input that layer saw)^T × (the blame that reached it); subtract, from a random start, and XOR is learned from nothing.
8. **Chapters 7–8.** Depth reuses folds, so a few small layers beat one huge one; the hidden set has the last word on size; and Nova's million knobs are trained by the same four lines as the first one.

Explain it to a friend

Cover the right column. Answer out loud, in your own words. Then compare.

Question	A good short answer
How does a hidden layer solve XOR?	It re-maps the inputs into a new space where the two “yes” cases land together and away from the “no” cases, so one straight wall separates them.
Why must hidden neurons have an activation?	Without one, a weighted sum of weighted sums is a single weighted sum: $(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1)\mathbf{W}^2 + \mathbf{b}^2 = \mathbf{x}\tilde{\mathbf{W}} + \tilde{\mathbf{b}}$. Ten straight layers are one straight layer.
What does one hidden ReLU neuron do to the map?	Folds it along the line where its sum is zero. The output cuts the folded sheet with one straight line.
What is forward propagation, in one line?	For each layer, $\mathbf{Z} = \mathbf{A}\mathbf{W} + \mathbf{b}$, then $\mathbf{A} = g(\mathbf{Z})$; rows are examples, columns are neurons.
What is the blame of a hidden neuron?	The blame of the neuron it feeds × the weight of the wire between them × the slope of its own activation. Summed over all the neurons it feeds.
Why does blame go through $\mathbf{W}^T$?	Forwards, neuron j sends its output along row j of $\mathbf{W}$ to every neuron in the next layer. Backwards, it collects blame from all of them along the same row — a dot product with row j , which is a product with $\mathbf{W}^T$.
What is the gradient of a weight matrix?	(The input that layer saw) ^T × (the blame at that layer): blame × input, added over the batch. Same shape as the matrix.
Why must the start be random?	Identical hidden neurons receive identical blame and stay identical forever; a network of copies is one neuron. Small random values break the tie.
What is a dead ReLU?	A neuron whose sum is negative for every example: slope 0, so it receives no blame, so it never moves, so it stays dead.
Depth or width?	Both add hinges. Depth reuses them — each new layer folds the already-folded sheet — so it usually gets more shape per knob. It also makes blame travel through more slopes, which is why ReLU matters.
How do you check a hand-written backward pass?	Nudge each knob by $\pm\epsilon$, measure the loss, divide by 2ϵ , compare with backprop. They should agree to many decimals.
What does <code>loss.backward()</code> do in a framework?	Chapter 5, automatically: it records the forward pass and passes the blame back through every operation.

✦ QUESTIONS WORTH CARRYING (NO ANSWERS HERE — ON PURPOSE)

- 1 · The blame reaching a hidden neuron passes through the output's weights W^2 . Early in training those weights are small and random, so the hidden layer is being taught by an output that does not yet know anything. Why does this work at all — and what does that say about the order in which layers learn?
- 2 · Our hand-built XOR net and our learned XOR net used different features. If a network learns its own features from data, what do we lose compared with features a human designed — and how would you find out what a hidden neuron in a million-knob network has learned to see?
- 3 · Backpropagation multiplies slopes along a path. Suppose a network had a wire that skipped a layer, connecting layer 1 directly to layer 3 as well as through layer 2. Write the blame reaching layer 1. What problem from Chapter 7 does that extra wire help with? (This is the idea behind the “residual” networks that see.)
- 4 · Nothing in the backward pass needed the labels except the seed $(P - Y)/n$. What could you put in that seed instead of labels — another network's opinion, a reward, the input itself — and what would the network then be learning?

Words to know — the whole book

Hidden layer Neurons between input and output; they re-draw the map.	MLP / Feedforward net Layers of neurons, each fed by all of the previous layer; no loops.	Depth, width Number of hidden layers; neurons per layer.
Activation g The per-neuron bend: sigmoid, tanh, ReLU.	ReLU $\max(0, z)$; slope 1 on, 0 off; the modern default.	Pre-activation Z The weighted sums before the bend; $\mathbf{Z}^l = \mathbf{A}^{l-1}\mathbf{W}^l + \mathbf{b}^l$.
Forward propagation $\mathbf{Z} = \mathbf{A}\mathbf{W} + \mathbf{b}$, $\mathbf{A} = g(\mathbf{Z})$, layer by layer, whole batch at once.	Broadcasting Adding the bias row to every row of a matrix.	Matched pair Output + loss whose slopes cancel: blame = guess – truth.
Output blame $\mathbf{D}^L$ $(\mathbf{P} - \mathbf{Y})/n$; the seed of backpropagation.	Backpropagation Every neuron's blame from the output backwards; then blame $\times$ input.	Blame $\delta, \mathbf{D}$ $\partial L / \partial \mathbf{Z}$: how the loss changes if a neuron's sum nudges.
$\mathbf{W}^T$ The transpose; blame goes back through it.	$\odot$ Entry-by-entry multiplication, as in $(\mathbf{D}\mathbf{W}^T) \odot g'(\mathbf{Z})$.	Gradient check Backprop against nudging each knob by $\pm\epsilon$.
Initialisation Small random start, scaled by $1/\sqrt{\text{inputs}}$ ($\times\sqrt{2}$ for ReLU).	Symmetry breaking Why the start must be random: copies stay copies.	Plateau A flat stretch of the loss curve before the drop.
Local minimum Where every small change makes the loss worse.	Dead ReLU Off for every example; receives no blame; never recovers.	Vanishing gradient Blame fading through many small slopes; ReLU's cure.
Capacity How complicated a boundary the knobs could draw.	Over- / underfitting Following the noise / too simple for the shape.	Early stopping Stop when the hidden-test curve turns down.
Universal approximation Enough hidden neurons can fit any curve; depth does it cheaply.	Regulariser Weight decay, dropout, augmentation: pressure against memorising.	Adam, momentum Smarter “subtract” steps; the loop is unchanged.



The end — and the beginning

Nova hears its name. Every wall it draws can bend. Every knob it has was found from examples.

WHAT YOU CAN DO NOW

You can take a table of numbers with answers, decide whether it asks “how much”, “yes or no” or “which one”, choose the matched output and loss, build a network of the right shape, start it randomly, run it forwards with three matrix products, send the blame backwards through the same wires, subtract, and watch two curves to know when to stop. You can do every one of those steps by hand on a tiny example, and you have code that does them at any size. You also know the traps — straight lines that collapse, neurons that die, gradients that vanish, networks that memorise — and what people do about them.

Everything you meet next — convolutions, attention, transformers, the models that see and speak — is these same pieces in new arrangements. When one of them looks like magic, find the forward pass, find the loss, and ask where the blame goes. It always goes back through the wires.

Deep Learning, Told Simply · a five-book story for the Deep Learning (DNN) course · Prof. Saurabh